

ESERCIZI D'ESAME SULLE QUERY - PROVE D'ESAME 2023-2024.

Corso di Basi di Dati.

Simone Remoli.

Esercizio 1 - In riferimento al seguente modello relazionale effettuare le seguenti query.

FORNITORE(ragione sociale, indirizzo, località, telefono, email, sito web*, rappresentante*, costo orario*)
ORARIO(fornitore, giorno, apertura, chiusura)
CATEGORIAFORNITORE(fornitore, categoria)
CATEGORIA(descrizione, categoria superiore)
UTENTE(username)
RECENSIONE(fornitore, utente, data, verificata, stelle, commento)

con i vincoli:

ORARIO(fornitore) $\subseteq$ FORNITORE(ragione sociale)
CATEGORIAFORNITORE(fornitore) $\subseteq$ FORNITORE(ragione sociale)
CATEGORIAFORNITORE(categoria) $\subseteq$ CATEGORIA(descrizione)
CATEGORIA(categoria superiore) $\subseteq$ CATEGORIA(descrizione)
RECENSIONE(fornitore) $\subseteq$ FORNITORE(ragione sociale)
RECENSIONE(utente) $\subseteq$ UTENTE(username)

1.1: Trovare i fornitori di servizi che abbiano una valutazione media maggiore di 4 stelle.

1.2: Trovare le località che non hanno alcun fornitore di servizi con recensioni verificate.

Svolgimento.

1.1

```
SELECT recensione.fornitore, AVG(recensione.stelle) AS  
valutazione_media  
FROM recensione  
GROUP BY recensione.fornitore  
HAVING AVG(recensione.stelle) > 4;
```

A mio modo di vedere, il join con fornitore_servizi (o fornitore) nella query del Prof. Pellegrini non è strettamente necessario per rispondere alla domanda, perché tutto quello che serviva — cioè i fornitori con media stelle > 4 — si poteva ottenere direttamente dalla tabella recensione.

1.2

```
SELECT DISTINCT fornitore.località  
FROM fornitore  
WHERE fornitore.località NOT IN (  
    SELECT DISTINCT fornitore.località
```

```

FROM recensione, fornitore
WHERE fornitore.ragione_sociale = recensione.fornitore
AND recensione.verificata = TRUE
);

```

Esercizio 2 - In riferimento al seguente modello relazionale effettuare la seguente query.

PALAZZETTO(Nome,Città,Capienza)
 INCONTRO(Nome Palazzetto,Data,Ora,Squadra1, Squadra2)
 NAZIONALE(Nazione,Continente,Livello)

2.1: Determinare le squadre che incontrano solo squadre dello stesso livello.

Svolgimento.

2.1

```

SELECT FIRST.squadra1, FIRST.squadra2, FIRST.livello
FROM (
  SELECT incontro.squadra1, incontro.squadra2, nazionale.livello
  FROM incontro, nazionale
  WHERE incontro.squadra1 = nazionale.nazione
) AS FIRST

```

INTERSECT

```

SELECT SECOND.squadra1, SECOND.squadra2, SECOND.livello
FROM (
  SELECT incontro.squadra1, incontro.squadra2, nazionale.livello
  FROM incontro, nazionale
  WHERE incontro.squadra2 = nazionale.nazione
) AS SECOND;

```

*È inteso, per me, che basta almeno una partita di pari livello e non voglio escludere le squadre che hanno anche partite con livelli diversi. Questa è la mia supposizione.
 Il Professore non ha questa supposizione, di seguito la sua query.*

Soluzione del Prof.Ing.A.Pellegrini

```
SELECT nazionale.nazione
FROM nazionale
WHERE NOT EXISTS (
    SELECT *
    FROM incontro, nazionale AS nazionale2
    WHERE incontro.squadra1 = nazionale.nazione
        AND incontro.squadra2 = nazionale2.nazione
        AND nazionale.livello <> nazionale2.livello
)
AND NOT EXISTS (
    SELECT *
    FROM incontro, nazionale AS nazionale2
    WHERE incontro.squadra2 = nazionale.nazione
        AND incontro.squadra1 = nazionale2.nazione
        AND nazionale.livello <> nazionale2.livello
);
```

Ora procedo a spiegare la soluzione del professore che a primo impatto può sembrare estremamente complicata, ma che, nella realtà, è un risultato molto intuitivo.

Ma prima di cimentarmi nella spiegazione, bisogna delineare alcuni aspetti fondamentali sulla NOT EXISTS.

Di seguito una rappresentazione facile della NOT EXISTS.

NOT EXISTS (subquery) controlla se la sottoquery **non restituisce nemmeno una riga**.

NOTA SULLA NOT EXISTS.

```
SELECT nome
FROM Studenti
WHERE NOT EXISTS (
    SELECT *
    FROM Assenze
    WHERE Assenze.id_studente = Studenti.id
);
```

Out: Giulia.

NOT EXISTS in SQL serve per verificare se una sottoquery non restituisce nemmeno una riga.

Studenti		
id	nome	classe
1	Anna	1A
2	Luca	1B
3	Giulia	1A

Assenze	
id_studente	data
1	2025-05-01
1	2025-05-02
2	2025-05-01

*Per ogni studente, cerca nelle assenze.
 Se non trova nulla allora lo studente viene tenuto.
 Se trova almeno una riga allora lo studente viene scartato.*

In sostanza, al bar si può dire: *"se la subquery non tira fuori niente, tieni la riga esterna".*

E la tabella **studenti** è quella che produce le righe esterne.

Quindi, ora vediamola passo passo.

Iterazione esterna → Nome = Anna (id = 1).

La sottoquery è:

```
SELECT *
FROM Assenze
WHERE Assenze.id_studente = 1;
```

E questa restituisce qualcosa.

Quindi Anna viene scartata.

Iterazione esterna → Nome = Luca (id = 2).

La sottoquery è:

```
SELECT *
FROM Assenze
WHERE Assenze.id_studente = 2;
```

E questa restituisce qualcosa.

Quindi Luca viene scartato.

Iterazione esterna → Nome = Giulia (id = 3).

La sottoquery è:

```
SELECT *
FROM Assenze
WHERE Assenze.id_studente = 3;
```

E questa NON restituisce NULLA.
Quindi Giulia viene TENUTA.

Da non confondere con la NOT IN.
NOT IN - ESEMPIO.

```
SELECT nome
FROM studenti
WHERE classe NOT IN
(
    SELECT classe
    FROM classi_bloccate
);
```

Regola: la subquery deve restituire una colonna sola!
 Il numero di righe non conta, il tipo di dato della colonna che restituisce la subquery deve essere compatibile con la colonna esterna.

Ora riprendiamo la Query del Professore e quindi anche la NOT EXISTS.

*Ora supponiamo di avere queste tabelle **incontro** e **nazionale**:*

A-Z squadra1 ▼	A-Z squadra2 ▼	🕒 data ▼	A-Z palazzetto ▼
📄 Italia	📄 Germania	2025-05-01	Arena A
📄 Germania	📄 Argentina	2025-05-03	Arena C
📄 Brasile	📄 Argentina	2025-05-04	Arena D
📄 Canada	📄 Canada	2025-05-05	Arena E
📄 Italia	📄 Brasile	2025-09-09	Arena B

A-Z nazione ▼	A-Z continente ▼	123 livello ▼
📄 Argentina	SudAmerica	2
Brasile	SudAmerica	2
Canada	America	3
Germania	Europa	1
Italia	Europa	1

*La query del Professore restituisce: **Canada**.*

La nazionale Canadese è l'unica che affronta un avversario di pari livello.

Cominciamo ad eseguire la query passo passo.

```
SELECT nazionale.nazione
FROM nazionale
WHERE NOT EXISTS (
    SELECT *
    FROM incontro, nazionale AS nazionale2
    WHERE incontro.squadra1 = nazionale.nazione
        AND incontro.squadra2 = nazionale2.nazione
        AND nazionale.livello <> nazionale2.livello
)
AND NOT EXISTS (
    SELECT *
    FROM incontro, nazionale AS nazionale2
    WHERE incontro.squadra2 = nazionale.nazione
        AND incontro.squadra1 = nazionale2.nazione
        AND nazionale.livello <> nazionale2.livello
);
```

```
1.          nazionale.nazione = Italia. (Esterno)
          nazionale2.nazione = Italia. (Interno)
```

Prima subquery.

```
SELECT *
FROM incontro, nazionale AS nazionale2
WHERE incontro.squadra1 = 'Italia'
AND incontro.squadra2 = 'Italia'
AND nazionale.livello <> nazionale2.livello
```

La tabella che mi viene restituita è **nulla**: non ci sono match Italia vs Italia. Quindi per il momento Italia viene tenuta.

```
2.          nazionale.nazione = Italia. (Esterno)
          nazionale2.nazione = Germania. (Interno)
```

Prima subquery.

```
SELECT *
FROM incontro, nazionale AS nazionale2
WHERE incontro.squadra1 = 'Italia'
AND incontro.squadra2 = 'Germania'
AND nazionale.livello <> nazionale2.livello
```

La tabella restituita è nulla, quindi per il momento Italia viene tenuta. Questo perché Italia e Germania hanno lo stesso livello, ma se hanno lo stesso livello vengono scartati.

```
3.           nazionale.nazione = Italia. (Esterno)
           nazionale2.nazione = Brasile. (Interno)
```

Prima subquery.

```
SELECT *
FROM incontro, nazionale AS nazionale2
WHERE incontro.squadra1 = 'Italia'
AND incontro.squadra2 = 'Brasile'
AND nazionale.livello <> nazionale2.livello
```

La query RESTITUISCE qualcosa, esiste un match Italia vs Brasile e i due livelli delle nazionali sono diversi.
Italia non viene tenuta.

Potrei andare avanti con `nazionale2.nazione = *` (tutte le altre squadre) mantenendo `nazionale.nazione = Italia`. Però è inutile! I match con l'Italia (come squadra1) sono finiti.

```
4.           nazionale.nazione = Germania. (Esterno)
           nazionale2.nazione = Argentina. (Interno)
```

Prima subquery.

```
SELECT *
FROM incontro, nazionale AS nazionale2
WHERE incontro.squadra1 = 'Germania'
AND incontro.squadra2 = 'Argentina'
AND nazionale.livello <> nazionale2.livello
```

La query RESTITUISCE qualcosa, esiste un match Germania vs Argentina e i due livelli delle nazionali sono diversi.
Germania non viene tenuta.

```
5.           nazionale.nazione = Brasile. (Esterno)
           nazionale2.nazione = Argentina. (Interno)
```

Prima subquery.

```
SELECT *
FROM incontro, nazionale AS nazionale2
WHERE incontro.squadra1 = 'Brasile'
AND incontro.squadra2 = 'Argentina'
AND nazionale.livello <> nazionale2.livello
```

La query NON restituisce NULLA, hanno lo stesso livello.

6. `nazionale.nazione = Canada. (Esterno)`
`nazionale2.nazione = Canada. (Interno)`

Prima subquery.

```
SELECT *  
FROM incontro, nazionale AS nazionale2  
WHERE incontro.squadra1 = 'Canada'  
AND incontro.squadra2 = 'Canada'  
AND nazionale.livello <> nazionale2.livello
```

La query NON restituisce NULLA, hanno lo stesso livello.

Quindi.

Per ogni Nazionale:

*Italia,
Germania,
Brasile,
Canada,
Argentina,*

Abbiamo verificato, con il primo NOT EXISTS, se la query restituisce qualcosa.

La query restituisce qualcosa quando nazionale.nazione = Italia, Germania.

Quindi la query non restituisce nulla quando nazionale.nazione = Brasile, Canada e Argentina.

Pertanto, il primo output con il primo filtro not exists è il seguente:

A-Z nazione ▼
Argentina
Brasile
Canada

Ora però c'è il secondo filtro.

Delle squadre 1 ho tolto quelle che fanno match con squadre di livello diverso: Italia e Germania.

*Ora applichiamo lo stesso procedimento per le squadre 2.
E tolgo Argentina e Brasile dall'insieme di tutte le squadre.*

A-Z nazione ▼
Canada
Germania
Italia

Quelli che stiamo vedendo nelle due immagini (Canada, Germania, Italia / Argentina, Brasile, Canada) non sono i risultati grezzi delle sottoquery “stampate da sole”, ma i risultati filtrati dalla query principale con il NOT EXISTS già applicato su ciascun blocco.

A-Z nazione ▼		A-Z nazione ▼
Argentina		Canada
Brasile	Intersezione	Germania
Canada		Italia

*Il risultato della query completa è **Canada**.*

Quello che stiamo vedendo è già il risultato delle condizioni con i NOT EXISTS. La query completa fa automaticamente l'intersezione di chi passa entrambi i filtri. Canada è l'unico sopravvissuto.

Esercizio 3 - In riferimento al seguente modello relazionale effettuare la seguente query.

MOTO(<u>Targa</u> , Cilindrata, Marca, Nazione, Tasse)
PROPRIETARIO(<u>Nome</u> , <u>Targa</u>)

3.1: Determinare i nomi delle persone che possiedono solo moto giapponesi di almeno due marche diverse.

3.2: Determinare le tasse che ogni proprietario deve pagare per le moto possedute (si assuma che una persona possa possedere più moto, ma che ogni moto sia posseduta da una sola persona).

Svolgimento.

3.1

```
SELECT PROPRIETARIO.Nome  
FROM PROPRIETARIO, MOTO  
WHERE PROPRIETARIO.Targa = MOTO.Targa  
AND MOTO.Nazione = 'Giappone'  
GROUP BY PROPRIETARIO.Nome  
HAVING COUNT(DISTINCT MOTO.Marca) >= 2;
```

Per ciascun gruppo, **HAVING COUNT(DISTINCT MOTO.Marca) >= 2** conta quante marche diverse ci sono, senza doppi

3.2

Questa è veramente facile.

```
SELECT PROPRIETARIO.Nome, SUM(MOTO.Tasse)  
FROM MOTO, PROPRIETARIO  
WHERE MOTO.Targa = PROPRIETARIO.Targa  
GROUP BY PROPRIETARIO.Nome;
```

Esercizio 4 - In riferimento al seguente modello relazionale effettuare la seguente query.

PERSONE(CF, Nome, Cognome, ComuneNascita, AnnoNascita, Sesso)
ASSESSORI(CF, Comune, DataInizio, DataFine)

con i vincoli:

ASSESSORI(CF) $\subseteq$ PERSONE(CF)

L'attributo Sesso è di tipo char(1) e può assumere solo i valori F (femmina) e M (maschio). L'attributo DataFine ha valore nullo nel caso in cui l'assessore sia ancora in carica.

4.1: Per ogni persona (con nome, cognome e codice fiscale) che ha ricoperto (o ricopre) almeno una volta il ruolo di assessore, trovare l'età alla quale ha ricoperto tale ruolo per la prima volta.

Svolgimento.

4.1

```
SELECT ASSESSORI.CF, PERSONE.Nome, PERSONE.Cognome,  
EXTRACT(YEAR FROM MIN(ASSESSORI.DataInizio)) - PERSONE.AnnoNascita  
FROM PERSONE, ASSESSORI  
WHERE PERSONE.CF = ASSESSORI.CF  
GROUP BY ASSESSORI.CF, PERSONE.Nome, PERSONE.Cognome, PERSONE.AnnoNascita;
```

SQL vuole che tutto ciò che non è dentro un aggregato (es. MIN()) compaia nel GROUP BY.

La versione del professore filtra direttamente le righe della tabella

e lavora solo sulla riga corretta, senza aggregati.

```
SELECT PERSONE.CF, PERSONE.Nome, PERSONE.Cognome, EXTRACT(YEAR
FROM ASSESSORI.DataInizio) - PERSONE.AnnoNascita
FROM PERSONE, ASSESSORI
WHERE PERSONE.CF = ASSESSORI.CF
AND ASSESSORI.DataInizio <= ALL
(
    SELECT DataInizio
    FROM ASSESSORI
    WHERE ASSESSORI.CF = PERSONE.CF //indice
);
```

La subquery è correlata alla riga esterna: per ogni CF, costruisce il suo sottoinsieme di date.

La prima volta nella subquery ho che PERSONE.CF=A1.

Quindi nella subquery escono tutte le date di inizio di ciascun codice fiscale.

La subquery viene eseguita una volta per ogni riga esterna.

CF	AnnoNascita	Sesso	... Nome,cognome..
1	2000	..	..
2	1999		
3	2001		
4	1985		
5	1899		
6	1991		

CF	Comune	DataInizio	DataFine
1	..	2021	..
1		2020	
2		2019	
2		2015	
3		2022	
4		2006	
3		2006	
3		2007	

Si fa una join quindi si prende tutto.

CF	Nome	Cognome	AnnoNascita	Sesso	DataInizio	DataFine	Età
1			2000		2021		
1			2000		2020		20
2			1999		2019		
2			1999		2015		16
3			2001		2022		
3			2001		2006		5
3			2001		2007		
4			1985		2006		21

Di questa proiezione alcuni campi.

*Per ogni CF = 1,2,3,4 la subquery restituisce le date di inizio.
(vedi sopra).*

DataInizio
2021
2020
2019
2015
2022
2006
2006
2007

E ora filtro: la data di inizio della tabella completa della join ha il campo DataInizio, e con questo campo ora controlliamo il più piccolo per ogni CF, quindi Assessori.DataInizio = 2020,2015,2006,2006.

Solo delle strisce verdi prendo l'età.

Si prendono solo le righe dove DataInizio è minore o uguale a tutte le altre date disponibili per lo stesso CF.

Esercizio 5 - In riferimento al seguente modello relazionale effettuare la seguente query.

CONDOMINI(Nome,Indirizzo)

SCALE(Codice,Condominio,QuotaMillesimi)

APPARTAMENTI(Interno,Scala,Condominio,QuotaMillesimi, Proprietario)

PROPRIETARI(CF, Nome, Cognome, Indirizzo)

SPESECONDOMINIALI(Numero,Data, Importo,Condominio)

SPESEDISCALA(Numero,Data, Importo,Scala,Condominio)

SPESEDIAPPARTAMENTO(Numero,Data, Importo,Interno,Scala,Condominio)

PAGAMENTI(Interno,Scala,Condominio,Data,Importo)

Vincoli:

SCALE(Condominio) $\subseteq$ CONDOMINI(Nome)

APPARTAMENTI(Scala,Condominio) $\subseteq$ SCALE(Codice,Condominio)

APPARTAMENTI(Proprietario) $\subseteq$ PROPRIETARI(CF)

SPESECONDOMINIALI(Condominio) $\subseteq$ CONDOMINI(Nome)

SPESEDISCALA(Scala,Condominio) $\subseteq$ SCALE(Codice,Condominio)

SPESEDIAPPARTAMENTO(Interno,Scala,Condominio) $\subseteq$ APPARTAMENTI(Interno,Codice,Condominio)

PAGAMENTI(Interno,Scala,Condominio) $\subseteq$ APPARTAMENTI(Interno,Codice,Condominio)

5.1: Trovare l'importo totale dei pagamenti (sinonimo di spese) per ciascuna scala del condominio "Le terrazze" che tenga conto sia delle spese relative alla scala che della quota relativa alle spese di condominio.

Svolgimento.

5.1

```
SELECT SCALE.Codice,  
       SUM(SPESEDISCALA.Importo) +  
       SUM(SPESECONDOMINIALI.Importo * SCALE.QuotaMillesimi /  
1000) AS ImportoTotale  
FROM SCALE, CONDOMINI, SPESEDISCALA, SPESECONDOMINIALI  
WHERE SCALE.C Condominio = CONDOMINI.Nome  
      AND SPESEDISCALA.Scala = SCALE.Codice  
      AND SPESEDISCALA.C Condominio = SCALE.C Condominio  
      AND SPESECONDOMINIALI.C Condominio = CONDOMINI.Nome  
      AND CONDOMINI.Nome = 'Le terrazze'  
GROUP BY SCALE.Codice;
```